

מערכות הפעלה - מטלה 1

ת"ז: 211544960, 212214316

TQ1

- א. מכיוון שאנו עובדים עם core אחד, לא יכולים לרוץ threads במקביל. בנוסף, לא קיימות פעולות I/O וה-context-switch זניח, לכן לא מתווסף זמן לריצת הפעולה. הפעולה מורכבת מ-3 threads באורך 40 מילי-שניות כל אחד, לכן זמן הריצה של הפעולה יהיה $120 = 40 \cdot 3$ מילי-שניות.
- ב. בשאלה זו יש לנו 3 cores לכן שלושת ה-threads יכולים לרוץ במקביל, לכן זמן הריצה יהיה $40 = \frac{40 \cdot 3}{3}$ מילי-שניות.
- ג. בשאלה זו יבוצע context-switch של 0.2 מילי-שניות בכל 5 מילי-שניות. בסעיף א' חישבנו שזמן הריצה של פעולה זו ב-core יחיד יהיה 120 מילי-שניות, לכן נוסיף לזמן זה מכפלה של מספר פעולות ה-context-switch ואורך פעולה בודדת – לכן זמן הריצה של פעולה זו יהיה $124.8 = 120 + 0.2 \cdot \frac{120}{5}$ מילי-שניות.

TQ2

- א. ערכי המשתנה האפשריים הם 1,2.
- ב. מקרה ראשון (בסיום $x=1$):
- thread A מתחיל $x = 0 \rightarrow t_a = x = 0$ context-switch ל-B $x = 0 \rightarrow t_b = x = 0$
- context-switch ל-A $x = 1 = (t_a + 1) \rightarrow t_a = 1$ context-switch ל-B $x = 1 = (t_b + 1) \rightarrow t_b = 1$
- $x = t_b = 1$
- מקרה שני (בסיום $x=2$):
- thread A מתחיל $x = 0 \rightarrow t_a = x = 0$ context-switch ל-B $x = 1 = (t_a + 1) \rightarrow t_a = 1$ context-switch ל-A $x = 1 = (t_b + 1) \rightarrow t_b = 1$
- context-switch ל-B $x = 2 = (t_b + 1) \rightarrow t_b = 2$ context-switch ל-A $x = 2 = (t_a + 1) \rightarrow t_a = 2$
- $x = t_b = 2$

TQ3

- א. ה-lock הפועל בשיטת FIFO הוא ticket-lock.
- בכניסת כל thread ל-critical section מתבצעת בדיקה של $curr_ticket == my_ticket$. מהיות ה- $curr_ticket$ עולה (ב-1) בכל הפעלה של הפעולה אזי מספר ה-ticket יושפע מסדר הכניסה. לדוגמה – thread_A נכנס לפני thread_B – אזי ה- my_ticket של thread_A יהיה קטן ממש מ- my_ticket של thread_B.
- מהיות הפעולה מעלה ב-1 בכל הפעלה שלה, אזי הפרש הכרטיסים יהיה 1 בין כל מספר, לכן לפני שthread ייקרא - ייקראו כל ה-threads עם ticket קטן ממנו, כלומר כאלו שנכנסו ל-critical section לפניו.

- כמו כן, לכל מספר כרטיס יכול להיות משויך thread אחד ולהפך, אזי בבדיקה $my_ticket == curr_ticket$, יכול להיבחר thread אחד בכניסה ל-lock, לכן ייכנס ל-lock זה thread אחד בכל פעם. לכן מדובר ב-lock שמבטיח עלייה מונוטונית בכניסה יחידנית, לכן הוא פועל בשיטת FIFO.
- ב. גם כאן, ה-lock שמבטיח bounded waiting הוא ticket-lock.
- יהי thread_A עם $my_ticket = k$. יהי c מספר טבעי כך ש- $c < k$ ו- $curr_ticket = c$. נשים לב כי על מנת ש-thread_A יקבל זמן CPU יידרשו לעבור k-c threads, לכן החסם העליון של זמן ההמתנה הוא k-c מעברים, לכן זמן ההמתנה של ה-thread חסום בזמן סופי, לכן קיים bounded waiting.
- ג. נשים לב כי ב-TAS lock אין תור או סדר בין התהליכים שמחכים.
- כל thread מנסה שוב ושוב לבצע test-and-set, וברגע שה-lock "משתחרר" - מי שיצליח "לתפוס" אותו ראשון ייכנס. לכן, ייתכן מצב שבו thread מסוים יחכה הרבה זמן, כיוון שייתכן ולא יתפוס את ה-lock ה-"משוחרר" מספר ניסיונות ברצף.
- במקרה כזה, אותו thread יכול להמשיך ולא להיכנס, למרות שהמנעול משתחרר כל פעם, לכן TAS לא מבטיח bounded waiting, ולכן עלול לגרום ל-starvation.

TQ4

- א. אלווריתם זה יכול להיכנס ל-deadlock, כיוון שתהליך יכול להחזיק את ה-mutex בלי לבדוק אם הוא יכול לבצע את הפעולה.
- נסתכל על דוגמה בה האלווריתם נכנס ל-deadlock:
- נניח שהבאפר ריק בתחילת הריצה – לכן $empty = CAP$, $full = 0$.
- נניח שה-consumer רץ ראשון. הוא מבצע פעולת acquire mutex ולאחר מכן $wait(full)$, אך כרגע $full = 0$ לכן אין פריטים בבאפר, לכן ה-consumer נחסם ע"י $wait(full)$ בזמן שהוא מחזיק את ה-mutex, עד ש-full יגדל.
- כעת נניח שה-producer מנסה לרוץ, הוא מנסה לבצע את acquire mutex - אך הוא לא יכול לבצע אותה כיוון שה-mutex מוחזק ע"י ה-consumer, לכן גם ה-producer מחכה ל-mutex.
- במצב זה ה-consumer מחזיק את ה-mutex עד ש-full יגדל, אך מי שיכול לגרום לו לגדול הוא ה-producer בעזרת פעולת $signal(full)$ שמחכה שה-mutex "ישתחרר" – לכן אנחנו נכנסים ל-deadlock.
- ב. השינוי המינימלי שיגרום לאלגוריתמים לעבוד כלומר שיהיה deadlock-free, הוא לבצע החלפה בין שלבים 1 ו-2, גם באלגוריתם ה-producer וגם באלגוריתם ה-consumer.
- נשים לב כי פתרון זה תקין מהיות השמת ההמתנה לפני השגת ה-mutex יוצרת ביטול של הבעיה הקודמת, לכן לא יהיה מצב של deadlock לאף אחד מה-threads.